Reconocimiento de tramas I2C con analizador lógico: pruebas ACK/NACK, escaneo de direcciones y comandos del SSD1306 sobre Raspberry Pi Pico

Juan Pablo Correa Niño – William Enrique Galvis Correa

Programa de Ingeniería en Telecomunicaciones, Universidad Militar Nueva Granada, Bogotá, Colombia
Informe de laboratorio, Comunicaciones Digitales (docente: José Rugeles)

***Resumen*—Se analizan tramas I2C capturadas con un analizador lógico (Logic 2) sobre un bus formado por una Raspberry Pi Pico con MicroPython y una pantalla OLED SSD1306 en la dirección 0x3C. Se identifican los elementos de la trama (Start, dirección de 7 bits con R/W, noveno bit y Stop) en una transacción con ACK (dirección 0x3C) y otra con NACK (dirección 0x3D), y se mide una frecuencia de SCL de 83,333 kHz frente a los 100 kHz configurados. Se verifica con `i2c.scan()` que solo responde la dirección 0x3C y se analiza la trama de un comando (0x80, 0xAE) enviado al SSD1306. Los resultados concuerdan con la especificación del protocolo. Las tramas de las demás opciones del menú de la pantalla se presentan como valores teóricos derivados del código, porque no se dispone de su captura.**

***Palabras clave*—I2C, ACK, NACK, analizador lógico, SSD1306, Raspberry Pi Pico, MicroPython.**

I. Introducción

El bus I2C (*Inter-Integrated Circuit*) comunica a un maestro con uno o varios esclavos mediante dos líneas de colector abierto, SCL y SDA, mantenidas en alto por resistencias de pull-up [2]. Cada transacción se delimita con las condiciones de Start y Stop y se compone de bytes de ocho bits, cada uno seguido de un noveno bit de reconocimiento (ACK o NACK). Observar estas tramas con un analizador lógico permite relacionar las instrucciones de un programa con el comportamiento eléctrico del bus y con el protocolo. Este informe documenta las pruebas de la guía de laboratorio [1] realizadas con una Raspberry Pi Pico y una pantalla OLED SSD1306 [3]: la identificación de los elementos de la trama, la comparación entre una transacción con ACK y una con NACK, el descubrimiento de la dirección mediante `i2c.scan()` y el análisis de los comandos enviados a la pantalla. En cada prueba se distingue entre valores medidos en las capturas y valores teóricos o derivados del código, y se señalan los datos de los que no se dispone. Los decimales se escriben con coma.

II. Montaje y metodología

A. Montaje experimental

El dispositivo bajo prueba es una pantalla OLED con controlador SSD1306, cuya dirección de 7 bits es 0x3C o 0x3D según el pin SA0 [3]; la dirección real se determina en la Sección III-D. El maestro es una placa Raspberry Pi Pico programada en MicroPython (Pico 2 W según la guía [1]) y los programas se ejecutaron desde Thonny. Las Secciones III-A a III-D usan el bus I2C1 con SCL en GP15 y SDA en GP14, mientras que el programa de la Sección III-E declara el bus 0 con SCL en GP13 y SDA en GP12. El código base indica pull-ups externos de 4,7 a 10 kΩ hacia 3V3 en ambas líneas.

Las señales se adquirieron con Logic 2, con el canal D0 en SCL y el canal D1 en SDA, tierra común y el decodificador I2C activo, según el procedimiento de la guía [1]. La guía pide muestrear a una tasa no menor que $10\,f_{\mathrm{SCL}}$; la tasa usada no aparece en las capturas (Sección IV).

B. Estructura de la trama I2C

Con el bus en reposo, SCL y SDA permanecen en alto. Una transacción comienza con la condición de Start, una transición de SDA de alto a bajo con SCL en alto, y termina con la condición de Stop, una transición de SDA de bajo a alto con SCL en alto [2]. Entre ambas, el maestro envía bytes de ocho bits, primero el más significativo. Cada bit se fija en SDA mientras SCL está en bajo y se muestrea en el flanco de subida de SCL. El primer byte contiene la dirección A de 7 bits seguida del bit R/W (0 para escritura, 1 para lectura):

$$B = (A \ll 1) \mid \mathrm{R}/\overline{\mathrm{W}} \tag{1}$$

Para A = 0x3C y escritura resulta B = 0x78, y para A = 0x3D resulta B = 0x7A. En el noveno pulso de SCL el maestro libera SDA. Si el esclavo direccionado reconoce el byte, fuerza SDA a bajo (ACK); si ningún dispositivo responde, SDA permanece en alto por el pull-up (NACK). La Fig. 1 resume la secuencia para B = 0x78. A una frecuencia f_{SCL}, el campo de dirección de nueve bits debe durar aproximadamente

$$T_{\mathrm{dir}} \approx \frac{9}{f_{\mathrm{SCL}}} \tag{2}$$

III. Resultados y análisis

A. Transacción con ACK (dirección 0x3C)

El Listado 1 sondea la dirección 0x3C con `i2c.writeto(ADDR, b"", True)`. Con un búfer vacío, la llamada genera Start, el byte de dirección con R/W igual a 0, la lectura del bit de reconocimiento y Stop,

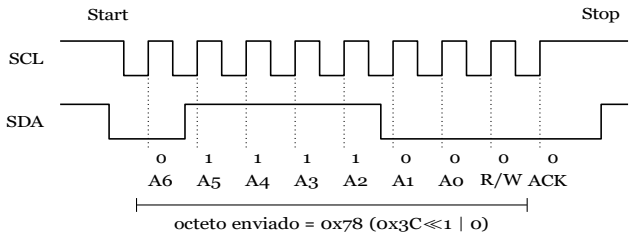
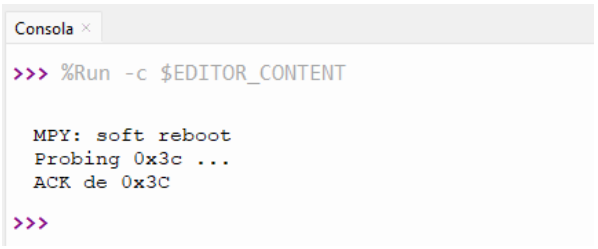
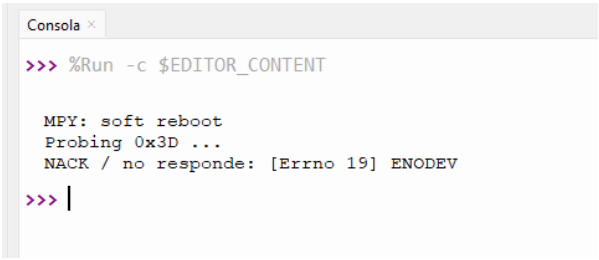


Fig. 1. Trama I2C de escritura a la dirección 0x3C (octeto 0x78) con ACK del esclavo. Esquema elaborado a partir de [1] y [2]; no es una captura.



(a)



(b)

Fig. 2. Consola de Thonny: (a) dirección 0x3C con ACK; (b) dirección 0x3D con NACK.

y lanza OSError si el esclavo no responde. La consola informa el ACK (Fig. 2a).

```
import machine, time
i2c = machine.I2C(1, scl=machine.Pin(15), sda=machine.Pin(14), freq
=100000)
ADDR = 0x3C
time.sleep_ms(1000)  # tiempo para armar el analizador
print("Probing 0x3c ...")
try:
    i2c.writeto(ADDR, b"", True)   # START, 0x3C(W), ACK, STOP
    print("ACK de 0x3C")
except OSError as e:
    print("NACK / no responde:", e)
```

Listado 1. OLED_ADDR_test.py. Para la prueba con NACK solo cambian ADDR = 0x3D y los textos impresos.

En la captura (Fig. 3) el decodificador etiqueta la transacción como *Setup Write to [0b 011 1100] + ACK*. El Start se detecta en t = 5,303619 s, el campo de dirección comienza en 5,303641 s y dura 110 µs, y el Stop se registra en 5,303774 s. Las flechas de muestreo del decodificador sobre los flancos de subida de SCL corresponden a los ocho bits del octeto 0x78, y el noveno pulso transporta el reconocimiento del esclavo.

Con los cursores se midió un periodo de SCL de 12 µs, con semiperiodo de 6 µs y ciclo útil de 50 %. Esto equivale a f_{SCL} = 83,333 kHz, valor que coincide con el mostrado

TABLA I
COMPARACIÓN ENTRE LAS CAPTURAS CON ACK Y CON NACK

Elemento	Captura ACK (0x3C)	Captura NACK (0x3D)
Start detectado (SDA↓ con SCL alto)	Sí, t = 5,303619 s	Sí, t = 6,844828 s
Octeto enviado (decodificador)	*Write to* [0b 011 1100], es decir, 0x78*	*Write to* [0b 011 1101], es decir, 0x7A*
Bit 9 (ACK = 0, NACK = 1)	ACK (0)	NACK (1)
Stop detectado (SDA↑ con SCL alto)	Sí, t = 5,303774 s	Sí, t = 6,844981 s
Frecuencia medida de SCL	83,333 kHz	83,333 kHz
Duración del campo de dirección	110 µs	110 µs
Duración Start–Stop*	155 µs	153 µs

*Valor derivado (dirección desplazada más R/W, o diferencia entre marcas de tiempo). Los demás valores se leen del decodificador o de los cursores.

por el decodificador. Según la ecuación (2), el campo de dirección debería durar 9 × 12 = 108 µs, consistente con los 110 µs medidos (2 µs de diferencia, cercanos a la resolución de 1 µs de las marcas de tiempo). La frecuencia medida es 16,7 % menor que los 100 kHz configurados en el Listado 1. La documentación de MicroPython describe el parámetro freq como la frecuencia máxima de SCL [4], por lo que una frecuencia real menor no contradice la configuración; la causa concreta de la diferencia no se investigó.

B. Transacción con NACK (dirección 0x3D)

Al cambiar ADDR a 0x3D, una dirección sin dispositivo, la consola informa NACK / no responde: [Errno 19] ENODEV (Fig. 2b): MicroPython traduce la ausencia de ACK en una excepción OSError con el código de dispositivo inexistente. En la captura (Fig. 4) el decodificador muestra *Setup Write to [0b 011 1101] + NAK*. El Start ocurre en 6,844828 s, el campo de dirección comienza en 6,844850 s con 110 µs de duración y el Stop se registra en 6,844981 s. En el noveno pulso SDA permanece en alto, y la captura resalta ese bit con la nota *NACK en bit 9*. Los cursores dan de nuevo un periodo de 12 µs (83,333 kHz). El octeto enviado es 0x7A.

C. Comparación entre ACK y NACK

Se mantiene la estructura completa de la trama: reposo, Start, ocho pulsos de datos y un noveno de reconocimiento, Stop, un campo de dirección de 110 µs y f_{SCL} = 83,333 kHz (Tabla I). Cambian únicamente el bit menos significativo de la dirección (0x3C es 011 1100 y 0x3D es 011 1101, es decir, los octetos 0x78 y 0x7A), el nivel de SDA en el noveno pulso (0 en el ACK y 1 en el NACK), la etiqueta del decodificador y el comportamiento del programa, que en el segundo caso recibe OSError. La diferencia de 2 µs entre las duraciones Start–Stop no se interpreta, por estar

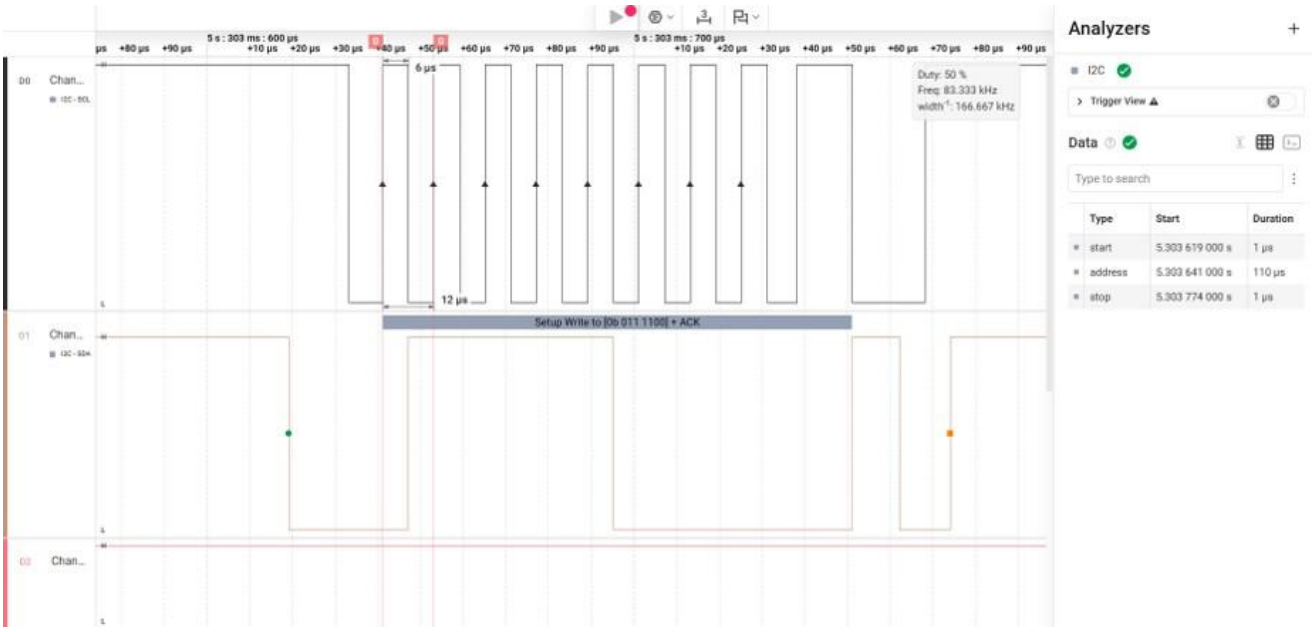


Fig. 3. Captura en Logic 2 de la transacción con dirección 0x3C: el decodificador indica *Setup Write to [ob 011 1100] + ACK*. Se observan los cursores usados para medir el periodo de SCL (12 µs).



Fig. 4. Captura en Logic 2 de la transacción con dirección 0x3D: el decodificador indica *Setup Write to [ob 011 1101] + NAK* y el noveno bit queda resaltado como NACK.

cerca de la resolución temporal de las marcas.

La dirección incorrecta no recibe ACK porque cada esclavo compara los siete bits recibidos con su propia dirección. En el SSD1306, seis de esos bits son fijos y el menos significativo lo fija el pin SA0, de modo que la dirección es 0x3C o 0x3D [3]. La pantalla del montaje responde en 0x3C (Sección III-D), así que ante 0x3D la comparación falla y el esclavo no maneja SDA en el noveno pulso. Como SDA es una línea de colector abierto, nadie la lleva a bajo y el pull-up la mantiene en alto, lo que el maestro interpreta como NACK.

D. Descubrimiento de la dirección con i2c.scan()

El Listado 2 crea el bus I2C1 sin fijar la frecuencia y ejecuta `i2c.scan()`, que sondea las direcciones de 7 bits de 0x08 a 0x77 con Start, dirección más R/W en escritura, lectura del ACK y Stop [1]. La consola lista únicamente 0x3c (Fig. 5).

```
1 import machine
2 i2c = machine.I2C(1, scl=machine.Pin(15), sda=machine.Pin(14))
3 devices = i2c.scan()
4 if devices:
5     for d in devices:
6         print(hex(d))
```

Listado 2. scan_i2c_addr.py.

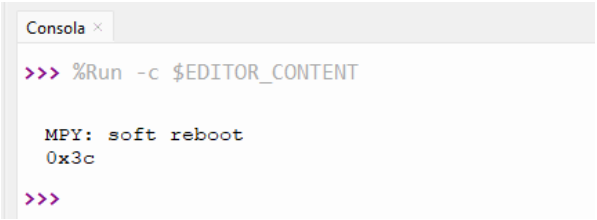


Fig. 5. Consola de Thonny tras ejecutar el escaneo: solo se detecta la dirección 0x3c.

TABLA II
SONDEOS VISIBLES EN LA CAPTURA DEL ESCANEO

Dirección	Bit 9	Inicio del campo (s)	Duración	Δt*
0x3A	NAK	3,626216	36 µs	—
0x3B	NAK	3,626273	36 µs	57 µs
0x3C	ACK	3,626330	36 µs	57 µs
0x3D	NAK	3,626392	36 µs	62 µs
0x3E	NAK	3,626449	36 µs	57 µs

*Diferencia con el inicio del sondeo anterior (valor derivado de las marcas de tiempo).

En las capturas (Figs. 6 y 7) cada dirección aparece como una transacción independiente, con su propio Start y su propio Stop, etiquetada *Write to* 0x*XX*. Entre las direcciones visibles, solo 0x3C recibe ACK (Tabla II), y la vista general muestra la secuencia de W[0x33] a W[0x3F] con W[0x3C] resaltada. Esto es coherente con la consola: al ser 0x3C la única dirección con ACK, es la única que devuelve el escaneo.

El campo de dirección dura 36 µs, es decir, 4 µs por bit, lo que equivale a f_{SCL} = 250 kHz y coincide con la medida que muestra la Fig. 7. Como el script no fija `freq`, se usó el valor por omisión del firmware, que se supone de 400 kHz [4] (no verificado). De ser así, la frecuencia medida sería 37,5 % menor que la solicitada, frente al 16,7 % de la Sección III-A, lo que sugiere que la diferencia no es un factor fijo. Los sondeos consecutivos se separan 57 µs, salvo el intervalo tras la dirección con ACK (de 0x3C a 0x3D), que es de 62 µs; no se determinó el origen de esos 5 µs adicionales. Si el intervalo de 57 µs se mantuviera para las 112 direcciones, el escaneo completo tomaría unos 6,5 ms (estimación, no medida).

E. Comandos hacia la pantalla SSD1306

El programa del menú envía los comandos con las funciones del Listado 3. `send_cmd_raw` antepone el byte de control 0x80 (Co = 1, D/C# = 0), que indica que el byte siguiente es un comando, y `send_data_raw` usa 0x40 (Co = 0, D/C# = 1), tras el cual todos los bytes de la transacción se interpretan como datos de la GDDRAM [1], [3]. Por eso cada comando ocupa una transacción propia, mientras que `write_data` envía en una sola transacción el byte 0x40 seguido de 16 bytes de datos (`CHUNK = 16`).

```
def send_cmd_raw(cmd):
    i2c.writeto(ADDR, bytes([0x80, cmd & 0xFF]))
    time.sleep_ms(2)
def send_data_raw(db):
    i2c.writeto(ADDR, bytes([0x40, db & 0xFF]))
    time.sleep_ms(2)
class ChunkedSSD1306_I2C(SSD1306_I2C):
    def write_data(self, buf):
        mv = memoryview(buf)
        for i in range(0, len(mv), CHUNK):
            part = bytes(mv[i:i+CHUNK])
            self.i2c.writeto(self.addr,
                             b'\x40' + part)
```

Listado 3. Funciones de envío del menú (extracto).

La Fig. 8 muestra una transacción completa hacia 0x3C: *Setup Write to [0x3C]* + *ACK*, *0x80* + *ACK* y *0xAE* + *ACK*. El primer byte (0x78 en el bus) direcciona la pantalla, el segundo es el byte de control 0x80 y el tercero, 0xAE, es el comando *Set Display OFF* [1], [3]. Los tres bytes fueron reconocidos por el SSD1306, lo que implica 27 pulsos de SCL (3 × 9) entre el Start y el Stop. La captura no permite distinguir entre la opción 1 (Apagar) y la opción 8 (Comando RAW con AE), porque ambas generan la misma trama.

La Tabla III resume la trama esperada para cada opción del menú. Solo la opción de apagado cuenta con captura; las demás tramas se derivan del código y se marcan como teóricas. Por ejemplo, el contraste 155 equivale a 0x9B y se envía en dos transacciones, porque el comando 0x81 (*Set Contrast Control*) es de dos bytes. Para el dato RAW, 0x7E ($0111\ 1110_2$) escribe una columna de ocho píxeles con los bits 1 a 6 encendidos. Un refresco completo de un búfer de 128×32 píxeles (512 bytes) implica 512/16 = 32 transacciones de datos, además de los comandos de direccionamiento que envía el método `show()` del controlador; este recuento se deriva del código y del driver estándar de MicroPython, que no se incluye entre los adjuntos.

La consola de la ejecución muestra el escaneo inicial (`['0x3c']`), el uso de la dirección 0x3C y la ejecución de las opciones 1 a 9 y F sin ningún mensaje de error. Como `writeto` lanza `OSError` cuando no hay ACK (Sección III-B), la ausencia de errores indica que la pantalla reconoció todas las transacciones. El cambio de frecuencia a 50 000, 100 000 y 400 000 Hz también se completó y el texto de demostración se envió en cada caso. Sin embargo, la consola no permite verificar el efecto visual ni la frecuencia real de SCL tras cada ajuste, para lo cual habría que medirla con el analizador.

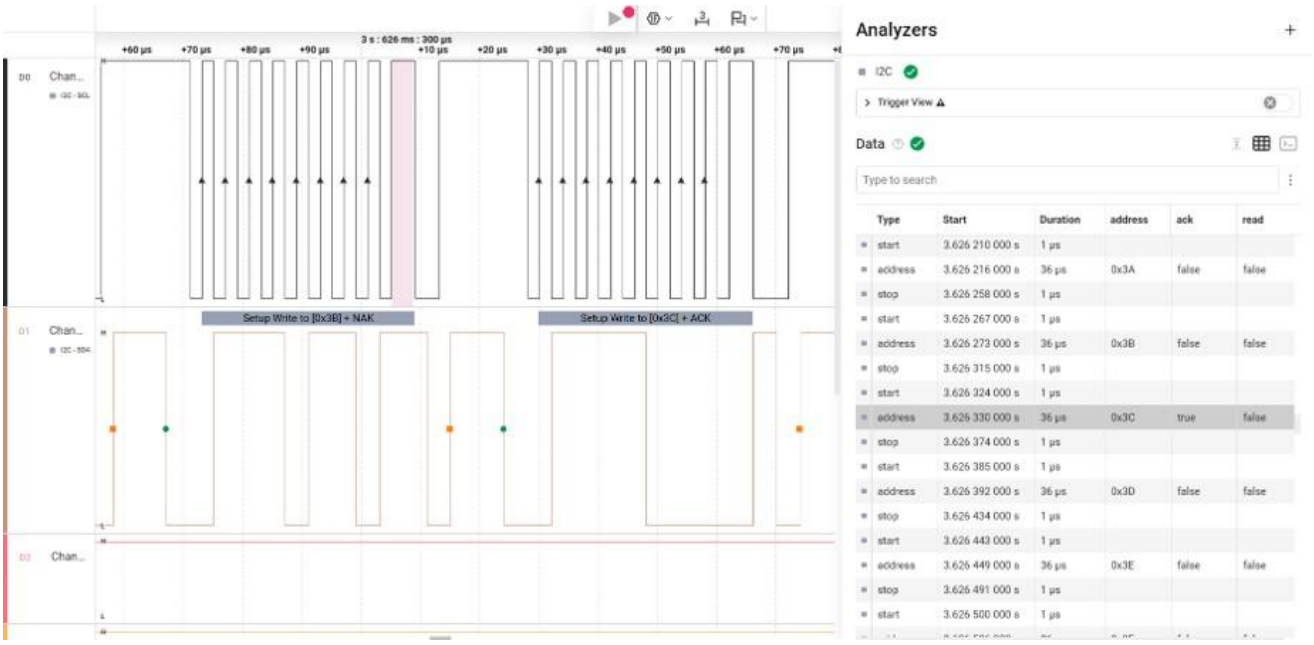


Fig. 6. Captura del escaneo en Logic 2 con la tabla de datos del decodificador: solo la dirección 0x3C tiene ACK, mientras que 0x3A, 0x3B, 0x3D y 0x3E responden NAK.



Fig. 7. Vista general del escaneo: sondeos consecutivos W[0x33] a W[0x3F]; el recuadro resalta W[0x3C], la única dirección con ACK. La medida que muestra la captura corresponde a 250 kHz.

IV. Alcance y datos pendientes

Los datos siguientes no están disponibles en las evidencias y no se completaron con estimaciones. La tasa de muestreo del analizador no aparece en ninguna captura. No hay capturas de las opciones 2 a 7, 9 y F del menú, por lo que sus tramas figuran como teóricas (Tabla III). La captura de la Fig. 8 no incluye una medida de f_{SCL} con cursores, ni se muestra a qué pines del bus 0 estaban conectados los canales del analizador en esa prueba. La captura con ACK conserva solo la etiqueta del decodificador y los cursores de medida, sin las anotaciones sobre Start, bits de dirección, R/W, ACK y Stop que pide la guía. Finalmente, no se determinó la causa de que f_{SCL} sea menor que la configurada.

V. Conclusiones

Las capturas permiten identificar los elementos de la trama I2C: Start, nueve pulsos de SCL para el octeto de dirección y su reconocimiento, y Stop. Con la dirección 0x3C el noveno bit es un ACK (octeto 0x78) y con 0x3D es

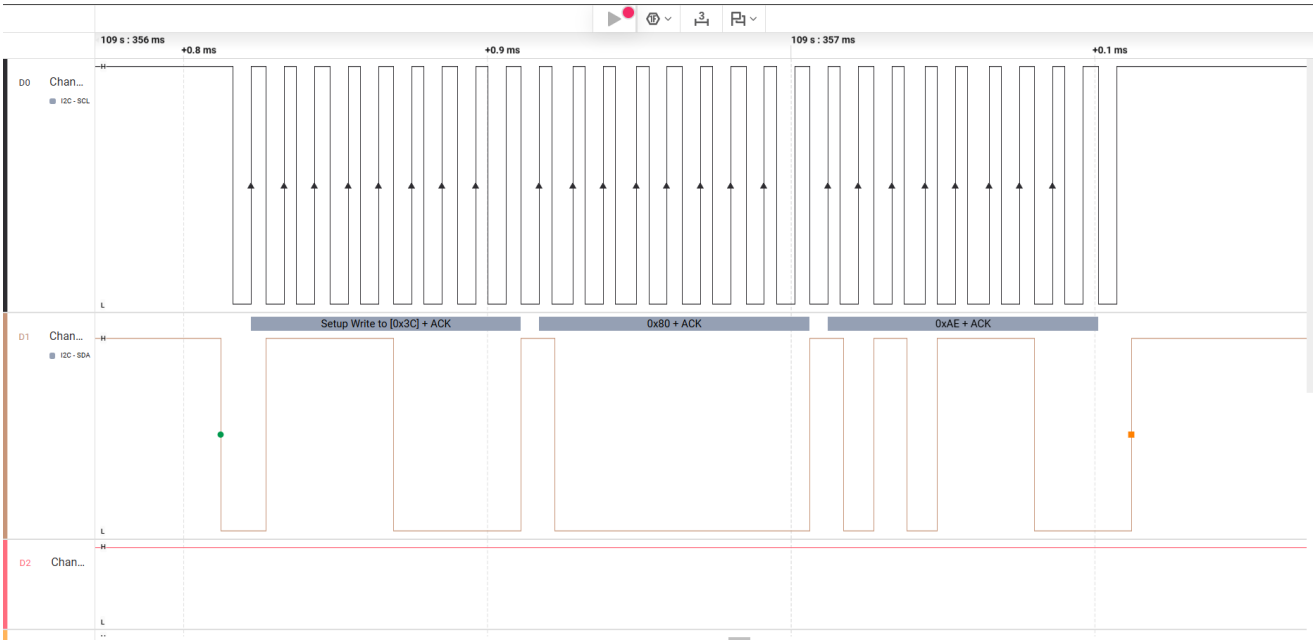


Fig. 8. Captura en Logic 2 del comando de apagado: dirección 0x3C con ACK, byte de control 0x80 con ACK y comando 0xAE con ACK.

TABLA III
TRAMA ESPERADA PARA CADA OPCIÓN DEL MENÚ DE LA PANTALLA

Opción del menú	Bytes tras Start y dirección 0x78 (cada transacción termina con Stop)
1. Apagar	0x80, 0xAE
2. Encender	0x80, 0xAF
3. Contraste (155)	Transacción 1: 0x80, 0x81. Transacción 2: 0x80, 0x9B
4. Invertir 1/0	0x80, 0xA7 (activar) o 0x80, 0xA6 (desactivar)
5–7. Limpiar, texto, animación	0x40 seguido de 16 bytes de datos, repetido 32 veces por refresco de 128×32
8. Comando RAW (AE)	0x80, 0xAE
9. Dato RAW (7E)	0x40, 0x7E
F. Cambiar frecuencia I2C	-

un NACK (octeto 0x7A), con la misma estructura y con la misma duración del campo de dirección de 110 µs. La ausencia de ACK se debe a que ningún esclavo coincide con la dirección enviada, y MicroPython la reporta como `OSError` con el código ENODEV.

El escaneo confirmó que la pantalla responde únicamente en 0x3C, y la traza muestra un sondeo independiente por dirección con ACK solo en 0x3C. La trama del comando de apagado (0x80, 0xAE) verifica el uso del byte de control y el reconocimiento de cada byte por el SSD1306. La frecuencia real de SCL resultó menor que la configurada en ambas pruebas (83,333 kHz frente a 100 kHz y 250 kHz frente al valor por omisión supuesto), un resultado que queda como punto por verificar con nuevas mediciones.

REFERENCIAS

[1] J. Rugeles, “Reconocimiento de tramas I2C con analizador lógico,” guía de laboratorio, Comunicaciones Digitales, Universidad Militar Nueva Granada, Bogotá, Colombia. Código: https://github.com/jrugeles/I2C.

[2] NXP Semiconductors, “UM10204: I2C-bus specification and user manual,” Rev. 7.0, oct. 2021.

[3] Solomon Systech, “SSD1306: 128 × 64 dot matrix OLED/PLED segment/common driver with controller,” hoja de datos.

[4] MicroPython, “class I2C – a two-wire serial protocol,” documentación de MicroPython. [En línea]. Disponible: https://docs.micropython.org/en/latest/library/machine.I2C.html